Ex 5 :

Demonstrate implementation of JWT Authentication for RESTful Web Service using Spring Security

Main.java :

1.com.cognizant.springlearn.filter:

AUTHCONTROLLER.JAVA :

```
 package com.cognizant.springlearn.controller;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.AuthenticationRequest;
import com.cognizant.springlearn.model.AuthenticationResponse;
import com.cognizant.springlearn.util.JwtUtil;

@RestController
public class AuthenticationController {

    private JwtUtil jwtUtil = new JwtUtil();

    @PostMapping("/authenticate")
    public AuthenticationResponse authenticate(
            @RequestBody AuthenticationRequest request) {

        String token = jwtUtil.generateToken(request.getUsername());

        return new AuthenticationResponse(token);
    }
}
```

HELLOCONTROLLER.JAVA

```
package com.cognizant.springlearn.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
```

```
    @GetMapping("/")
    public String home() {
        return "Welcome to Spring Security";
    }

    @GetMapping("/hello")
    public String hello() {
        return "Hello User";
    }
}
```

JWTAUTHORIZATIONFILTER.JAVA

```
package com.cognizant.springlearn.filter;

import java.io.IOException;


import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.web.filter.OncePerRequestFilter;

import com.cognizant.springlearn.util.JwtUtil;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class JwtAuthorizationFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil = new JwtUtil();

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain)
           throws ServletException, IOException {

        String header = request.getHeader("Authorization");
```

```
        if (header == null || !header.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = header.substring(7);

        try {

            String username = jwtUtil.validateToken(token);

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                            username,
                            null,
                            null);

            authentication.setDetails(
                    new WebAuthenticationDetailsSource()
                            .buildDetails(request));

            SecurityContextHolder.getContext()
                    .setAuthentication(authentication);

        } catch (Exception e) {

            SecurityContextHolder.clearContext();
        }

        filterChain.doFilter(request, response);
    }
}
```

AUTHENTICATIONREQUEST.JAVA :

```
package com.cognizant.springlearn.model;
public class AuthenticationRequest {
  private String username;
  private String password;
  public AuthenticationRequest() {
  }
  public AuthenticationRequest(String username, String password) {
     this.username = username;
     this.password = password;
  }
  public String getUsername() {
```

```
        return username;
    }
    public void setUsername(String username) {
        this.username = username;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

AUTHENTICATIONRESPONSE.JAVA

```
package com.cognizant.springlearn.model;
public class AuthenticationResponse {
    private String jwt;
    public AuthenticationResponse() {
    }
    public AuthenticationResponse(String jwt) {
        this.jwt = jwt;
    }
    public String getJwt() {
        return jwt;
    }
    public void setJwt(String jwt) {
        this.jwt = jwt;
    }
}
```

SECURITYCONFIG.JAVA

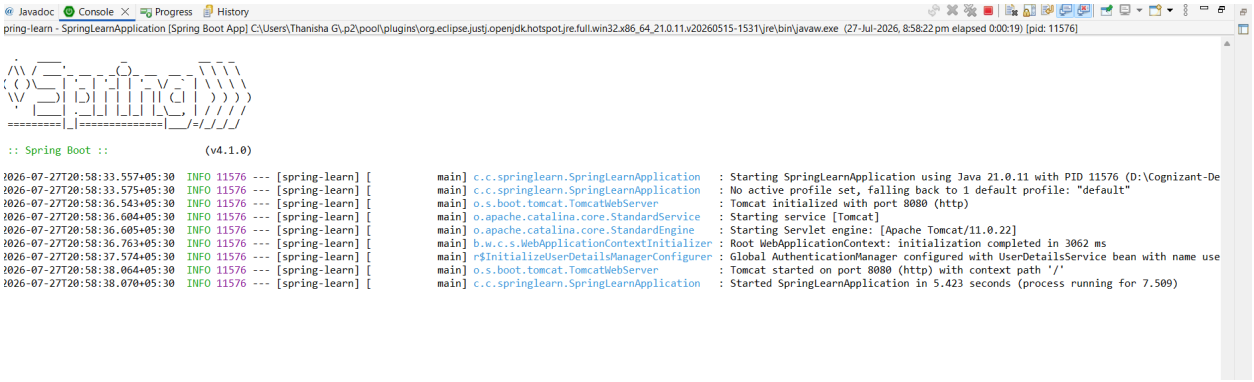
```
package com.cognizant.springlearn.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

```
import com.cognizant.springlearn.filter.JwtAuthorizationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                    .requestMatchers("/authenticate").permitAll()
                    .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults())
            .addFilterBefore(new JwtAuthorizationFilter(),
                    UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {

        UserDetails user = User.builder()
                .username("user")
                .password(passwordEncoder.encode("pwd"))
                .roles("USER")
                .build();

        UserDetails admin = User.builder()
                .username("admin")
                .password(passwordEncoder.encode("pwd"))
                .roles("ADMIN")
                .build();

        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

JWTUTIL.JAVA

```
package com.cognizant.springlearn.util;

import javax.crypto.SecretKey;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

public class JwtUtil {

    private static final SecretKey SECRET_KEY =
            Keys.hmacShaKeyFor(
                    "ThisIsASecretKeyForJwtGeneration123456789".getBytes());

    public String generateToken(String username) {

        return Jwts.builder()
                .subject(username)
                .signWith(SECRET_KEY)
                .compact();
    }

    public String validateToken(String token) {

        Claims claims = Jwts.parser()
                .verifyWith(SECRET_KEY)
                .build()
                .parseSignedClaims(token)
                .getPayload();

        return claims.getSubject();
    }
}
```

OUTPUT :


Javadoc
Console
Progress
History
pring-learn - SpringLearnApplication [Spring Boot App] C:\Users\Thanisha G\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (27-Jul-2026, 8:58:22 pm elapsed 0:00:19) [pid: 11576]
:: Spring Boot :: (v4.1.0)
2026-07-27T20:58:33.557+05:30 INFO 11576 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 21.0.11 with PID 11576 (D:\Cognizant-De
2026-07-27T20:58:33.575+05:30 INFO 11576 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-27T20:58:36.543+05:30 INFO 11576 --- [spring-learn] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-27T20:58:36.604+05:30 INFO 11576 --- [spring-learn] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-27T20:58:36.605+05:30 INFO 11576 --- [spring-learn] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-27T20:58:36.763+05:30 INFO 11576 --- [spring-learn] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 3062 ms
2026-07-27T20:58:37.574+05:30 INFO 11576 --- [spring-learn] [main] r$InitializeUserDetailsManagerConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name use
2026-07-27T20:58:38.064+05:30 INFO 11576 --- [spring-learn] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-07-27T20:58:38.070+05:30 INFO 11576 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 5.423 seconds (process running for 7.509)


localhost:8080/hello
School
Guidelines for Wikip...
All Bookmarks

Hello User